

# Data Preprocessing Pipeline

### Data Preprocessing: Foundation of Data Analysis & ML  
Transforming raw customer purchase data into clean, structured, and usable insights and ready for ML

Includes the following steps:

1. Load Data
2. Understand Dataset
3. Fix Column Names
4. Handle Duplicates
5. Handle Missing Values
6. Correct Datatypes
7. Fix Inconsistent Values
8. Detect Outliers (IQR Method)
9. Remove Outliers
10. Data Integration (Merging)
11. Data Reduction (Selecting Important Features)
12. Save Clean Dataset

## LOAD DATA Purpose: Import dataset into Pandas DataFrame

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv("Data_Preprocessing_customer_purchase.csv")
```

```
print("\n Original Data Sample:");
print(df.head());
```

|   | CustomerID | Gender | Age  | AnnualIncome | SpendingScore | Purchased |
|---|------------|--------|------|--------------|---------------|-----------|
| 0 | 1          | Female | 28.0 | 91748.0      | 69.0          | 0         |
| 1 | 2          | Female | 35.0 | 87860.0      | 57.0          | 0         |
| 2 | 3          | M      | 38.0 | 89635.0      | 44.0          | 1         |
| 3 | 4          | Female | NaN  | 89611.0      | 86.0          | 0         |
| 4 | 5          | Male   | 40.0 | 98985.0      | 57.0          | 0         |

# UNDERSTAND DATASET (Info + Stats + Missing Values)

Purpose: Get structure, datatypes, summary, missing data

```
print("\n Dataset Info:")
print(df.info());
print("-----")
print("\n Descriptive Statistics:")
print(df.describe());
print("-----")
print("\n Missing Values:");
print(df.isnull().sum());
```

## Dataset Info:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 51 entries, 0 to 50
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   CustomerID      51 non-null    int64
1   Gender          51 non-null    object
2   Age             48 non-null    float64
3   AnnualIncome    49 non-null    float64
4   SpendingScore   49 non-null    float64
5   Purchased       51 non-null    int64
dtypes: float64(3), int64(2), object(1)
memory usage: 2.5+ KB
None
```

## Descriptive Statistics:

|       | CustomerID | Age       | AnnualIncome | SpendingScore | Purchased |
|-------|------------|-----------|--------------|---------------|-----------|
| count | 51.000000  | 48.000000 | 49.000000    | 49.000000     | 51.000000 |
| mean  | 25.019608  | 39.270833 | 70090.428571 | 52.020408     | 0.411765  |
| std   | 14.833058  | 11.133120 | 24975.117477 | 26.891363     | 0.497050  |
| min   | 1.000000   | 19.000000 | 24466.000000 | 2.000000      | 0.000000  |
| 25%   | 12.500000  | 32.000000 | 46523.000000 | 31.000000     | 0.000000  |
| 50%   | 25.000000  | 38.500000 | 76623.000000 | 54.000000     | 0.000000  |
| 75%   | 37.500000  | 47.250000 | 91748.000000 | 69.000000     | 1.000000  |
| max   | 50.000000  | 59.000000 | 99769.000000 | 99.000000     | 1.000000  |

## Missing Values:

|              |   |
|--------------|---|
| CustomerID   | 0 |
| Gender       | 0 |
| Age          | 3 |
| AnnualIncome | 2 |

```
SpendingScore    2
Purchased         0
dtype: int64
```

## FIX COLUMN NAMES

Purpose: Remove spaces and standardize names

```
df.columns = df.columns.str.strip().str.replace(" ",
"_").str.capitalize()

print("\n Cleaned Column Names:");
print(df.columns.tolist());
```

```
Cleaned Column Names:
['Customerid', 'Gender', 'Age', 'Annualincome', 'SpendingScore',
'Purchased']
```

## HANDLE DUPLICATES : Purpose: Detect and remove duplicate rows

```
# Count duplicates
duplicate_count = df.duplicated().sum();

print(f"\n Number of Duplicate Rows Found: {duplicate_count}")
```

```
Number of Duplicate Rows Found: 1
```

```
# Show duplicate rows (if any)
duplicate_rows = df[df.duplicated()]

print("Show Duplicate Rows:- ", duplicate_rows);
```

```
Show Duplicate Rows:-      Customerid  Gender  Age  Annualincome
SpendingScore  Purchased
50              1  Female  28.0      91748.0          69.0          0
```

```
# Remove duplicates
if duplicate_count > 0:
    df = df.drop_duplicates()
    print("\nDuplicates removed successfully.")
else:
    print("\nNo action needed. Dataset is already clean.");
```

Duplicates removed successfully.

## HANDLE MISSING VALUES

Purpose: Fill missing values using defined logic

```
df['Annualincome'] =  
df['Annualincome'].fillna(df['Annualincome'].median())  
df['SpendingScore'] =  
df['SpendingScore'].fillna(df['SpendingScore'].median())  
  
#  
=====
```

```
df['Age'] = df['Age'].fillna(80)  
df['Annualincome'] = df['Annualincome'].fillna(90000)  
df['SpendingScore'] = df['SpendingScore'].fillna(50)  
print(df.columns);  
  
Index(['Customerid', 'Gender', 'Age', 'Annualincome', 'SpendingScore',  
      'Purchased'],  
      dtype='object')
```

## CORRECT DATATYPES

Purpose: Convert columns to correct datatypes

```
print("\nBefore Datatype Correction:")  
print(df[['Customerid', 'Purchased']].dtypes)  
  
df['Customerid'] = df['Customerid'].astype(int)  
df['Purchased'] = df['Purchased'].astype(int)  
  
print("\nAfter Datatype Correction:")  
print(df[['Customerid', 'Purchased']].dtypes)
```

Before Datatype Correction:

```
Customerid    int64  
Purchased     int64  
dtype: object
```

After Datatype Correction:

```
Customerid    int64
```

```
Purchased      int64
dtype: object
```

## FIX INCONSISTENT VALUES

Purpose: Standardize inconsistent text values (e.g., Gender)

```
print("\nBefore Fixing Gender Values:")
print(df['Gender'].unique());
```

```
Before Fixing Gender Values:
['Female' 'M' 'Male' 'F']
```

```
df['Gender'] = df['Gender'].replace({
    'M': 'Male',
    'F': 'Female',
    'male': 'Male',
    'female': 'Female',
    'MALE': 'Male',
    'FEMALE': 'Female'
});
```

```
print("\nAfter Fixing Gender Values:")
print(df['Gender'].unique());
```

```
After Fixing Gender Values:
['Female' 'Male']
```

## DETECT OUTLIERS (IQR METHOD)

Purpose: Identify extreme values that may affect model performance

```
def detect_outliers(col):
    # Calculate Q1 (25th percentile) → 25% of data lies below this
    value
    Q1 = df[col].quantile(0.25);

    # Calculate Q3 (75th percentile) → 75% of data lies below this
    value
    Q3 = df[col].quantile(0.75)

    # Calculate IQR (Interquartile Range) → difference between Q3 and
    Q1
    IQR = Q3 - Q1
```

```

    # Calculate lower bound → values below this are considered
    outliers # Set minimum limit
    lower = Q1 - 1.5 * IQR

    # Calculate upper bound → values above this are considered
    outliers # Set maximum limit
    upper = Q3 + 1.5 * IQR

    # Return rows where column values are either below lower bound OR
    above upper bound
    return df[(df[col] < lower) | (df[col] > upper)]

print("\n Outliers in Age:");

# Call the function for 'Age' column and print the detected outliers
print(detect_outliers('Age'));

```

```

Outliers in Age:
   Customerid  Gender  Age  Annualincome  Spendingscore  Purchased
3           4  Female  80.0         89611.0           86.0           0
10          11  Female  80.0         57580.0           15.0           0
25          26  Female  80.0         76623.0           13.0           0

```

## REMOVE OUTLIERS (IQR METHOD)

Purpose: Keep only normal values within IQR range

```

def remove_outliers(col):
    # Get 25% value (start range)
    Q1 = df[col].quantile(0.25)

    # Get 75% value (end range)
    Q3 = df[col].quantile(0.75)

    # Find middle range
    IQR = Q3 - Q1

    # Set minimum limit
    lower = Q1 - 1.5 * IQR

    # Set maximum limit
    upper = Q3 + 1.5 * IQR

    # Keep only values inside the range (remove outliers)
    return df[(df[col] >= lower) & (df[col] <= upper)]

# Remove outliers from Age column

```

```
df = remove_outliers('Age');
# Print final data size after removing outliers
print("\n After Removing Outliers:", df.shape)
```

After Removing Outliers: (47, 6)

## FEATURE ENGINEERING

Purpose: Create new useful features from existing data

```
df['Income_category'] = pd.cut(df['Annualincome'], bins=[0, 40000,
80000, 150000], labels=['Low', 'Medium', 'High'])

print("\nTop 5 Rows After Creating Income_category:\n");

print(df.head(5))

print("-----")

print(df['Income_category'].value_counts());
```

Top 5 Rows After Creating Income\_category:

|   | Customerid | Gender | Age  | Annualincome | Spending score | Purchased | \ |
|---|------------|--------|------|--------------|----------------|-----------|---|
| 0 | 1          | Female | 28.0 | 91748.0      | 69.0           | 0         |   |
| 1 | 2          | Female | 35.0 | 87860.0      | 57.0           | 0         |   |
| 2 | 3          | Male   | 38.0 | 89635.0      | 44.0           | 1         |   |
| 4 | 5          | Male   | 40.0 | 98985.0      | 57.0           | 0         |   |
| 5 | 6          | Male   | 35.0 | 75005.5      | 2.0            | 0         |   |

|   | Income_category |
|---|-----------------|
| 0 | High            |
| 1 | High            |
| 2 | High            |
| 4 | High            |
| 5 | Medium          |

```
-----
Income_category
Medium    20
High     19
Low       8
Name: count, dtype: int64
```

# DATA INTEGRATION (MERGE EXTRA DATA)

Purpose: Combine multiple datasets (joins)

```
extra_data = pd.DataFrame({
    'Customerid': [1, 5, 10],
    'City': ['Mumbai', 'Pune', 'Delhi']
});
```

```
print("\n Extra Data to Integrate:")
print(extra_data);
```

Extra Data to Integrate:

|   | Customerid | City   |
|---|------------|--------|
| 0 | 1          | Mumbai |
| 1 | 5          | Pune   |
| 2 | 10         | Delhi  |

```
df = pd.merge(df, extra_data, on='Customerid', how='left')
```

```
print("\n After Data Integration (Merged Dataset):")
```

```
print(df.head(5))
```

After Data Integration (Merged Dataset):

|   | Customerid | Gender | Age  | Annualincome | Spendingscore | Purchased | \ |
|---|------------|--------|------|--------------|---------------|-----------|---|
| 0 | 1          | Female | 28.0 | 91748.0      | 69.0          | 0         |   |
| 1 | 2          | Female | 35.0 | 87860.0      | 57.0          | 0         |   |
| 2 | 3          | Male   | 38.0 | 89635.0      | 44.0          | 1         |   |
| 3 | 5          | Male   | 40.0 | 98985.0      | 57.0          | 0         |   |
| 4 | 6          | Male   | 35.0 | 75005.5      | 2.0           | 0         |   |

|   | Income_category | City   |
|---|-----------------|--------|
| 0 | High            | Mumbai |
| 1 | High            | NaN    |
| 2 | High            | NaN    |
| 3 | High            | Pune   |
| 4 | Medium          | NaN    |

# DATA REDUCTION

Purpose: Keep only important ML columns

```
final_columns = [
    'Customerid', 'Age', 'Gender', 'Annualincome', 'Spendingscore',
    'Purchased', 'Income_category', 'City'
];
```



```
df_final = df[final_columns];
print("\n Final Clean Dataset Sample:\n", df_final.head());
```

```
Final Clean Dataset Sample:
  Customerid  Age  Gender  Annualincome  Spendingscore
Purchased \
0           1  28.0  Female      91748.0           69.0           0
1           2  35.0  Female      87860.0           57.0           0
2           3  38.0   Male      89635.0           44.0           1
3           5  40.0   Male      98985.0           57.0           0
4           6  35.0   Male      75005.5            2.0           0

Income_category  City
0           High  Mumbai
1           High    NaN
2           High    NaN
3           High   Pune
4          Medium    NaN
```

## SAVE CLEAN DATASET

Purpose: Export fully cleaned data for model training

```
df_final.to_csv("customer_purchase_cleaned_advanced.csv",
index=False);

print("\n COMPLETE DATA PREPROCESSING DONE SUCCESSFULLY!")
```

```
COMPLETE DATA PREPROCESSING DONE SUCCESSFULLY!
```